

VIETNAM NATIONAL UNIVERSITY-HO CHI MINH CITY

HO CHI MINH UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

KNOWLEDGE ENGINEERING DEPARTMENT

Midterm Test

Course: Introduction to Artificial Intelligence

Student:

23127165 - Nguyen Hai Dang

Instructors:

Mr Bui Tien Len

July 3, 2025



Contents

1	Abstract	2
2	Problem Definition	2
2.1	Selected Problem	2
2.2	Brainstorming Process	2
2.3	Explanation of Usage	3
3	Problem Formulation	4
3.1	Initial State	4
3.2	Actions	4
3.3	Transition Model	4
3.4	Goal Test	4
3.5	Path Cost	5
4	Methodology	5
4.1	Algorithm Category Selection	5
4.1.1	Comparison with Uninformed Search Algorithms	5
4.1.2	Comparison with Local Search Algorithms	6
4.1.3	Why Informed Search is Optimal for This Problem	7
4.2	Selection Within Informed Search	8
4.2.1	Greedy Best-First Search	8
4.2.2	A* Search	8
4.3	Algorithm Selection Summary	8
4.4	Heuristic Function Design	9
5	Results	9
5.1	Test Case 1 with Path Found	10
5.1.1	Problem Instance	10
5.1.2	Solve the Problem	11
5.1.3	Visualize the Search Process	13
5.2	Test Case 2 with No Path Found	15
5.2.1	Problem Instance	15
5.2.2	Solve the Problem	16
5.2.3	Visualize the Search Process	18
6	Analysis	20
6.1	Statistics Table: Test Case 1 (Path Found)	20
6.2	Statistics Table: Test Case 2 (No Path Found)	20
6.3	Performance Analysis	20
6.4	Optimality and Limitations	21
6.5	Challenges and Future Improvements	21
7	Conclusion	22

1 Abstract

This report presents a comprehensive analysis of applying search algorithms to solve *a robot navigation problem in a grid-based environment with obstacles*. The problem involves finding an optimal path for a robot to navigate from a starting position to a goal position while avoiding static obstacles and minimizing path cost. Two informed search algorithms were implemented and compared: **Greedy Best-First Search** and **A* Search**.

The study demonstrates the formulation of a real-world navigation scenario as a search problem, implementation of appropriate algorithms, and detailed performance analysis. Results show that A* Search provides optimal solutions with higher computational cost, while Greedy Best-First Search offers faster execution with potentially suboptimal solutions. The analysis includes comprehensive performance metrics, visual representations of search processes, and practical insights into algorithm selection for navigation problems.

2 Problem Definition

2.1 Selected Problem

The chosen problem named **Robot Navigation in Grid Environment** involves programming a robot to navigate through a 2D grid environment from a starting position to a target destination while avoiding obstacles. This is a fundamental problem in robotics and autonomous systems, with applications in warehouse automation, autonomous vehicles, and mobile robotics.

Problem Statement: Given a robot positioned at coordinates $(x_{\text{start}}, y_{\text{start}})$ in a 6x6 grid environment containing static obstacles, find an efficient path to reach the goal position at $(x_{\text{goal}}, y_{\text{goal}})$ while minimizing the total movement cost and avoiding collision with obstacles.

2.2 Brainstorming Process

For the brainstorming phase, a large language model used is **Gemini 2.5 Pro**

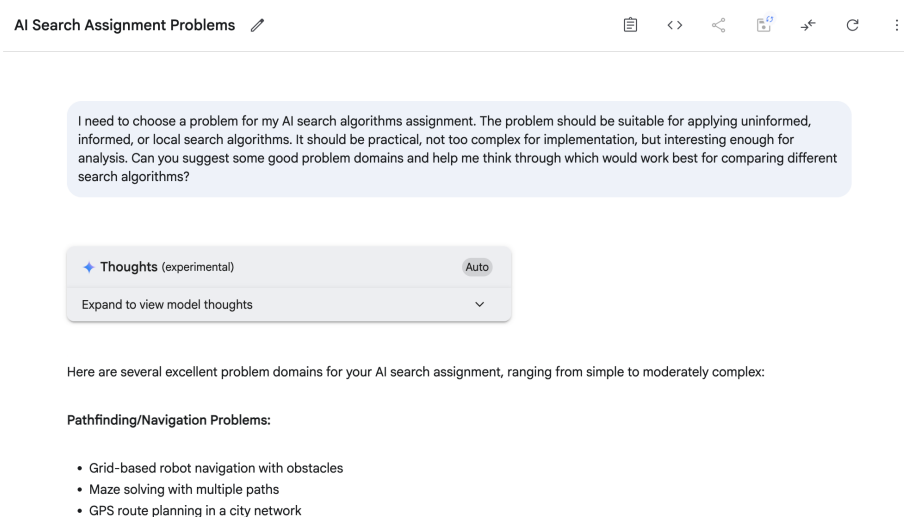


Figure 1: This is the selected prompt

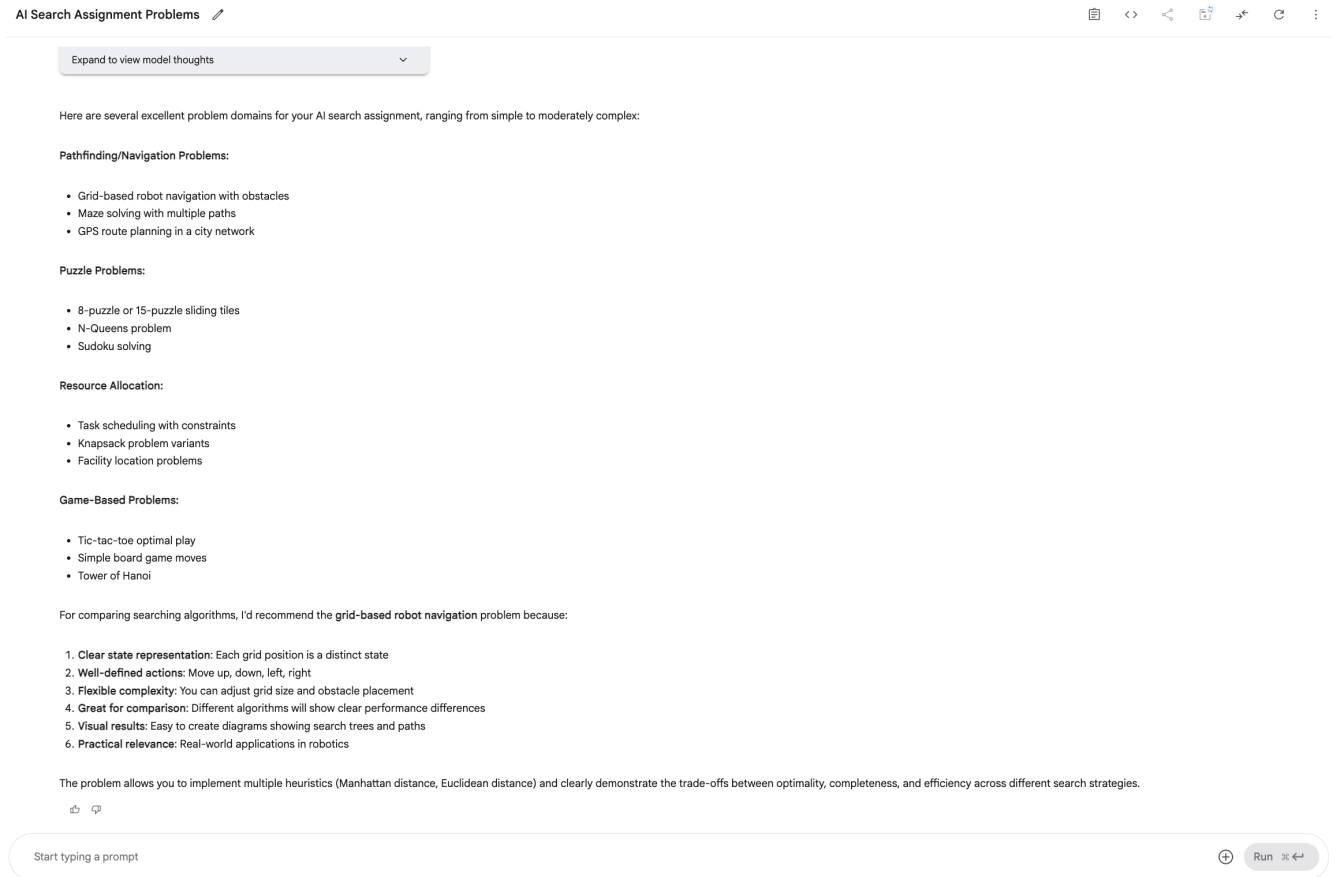


Figure 2: This is full the AI's response

2.3 Explanation of Usage

The AI's response was instrumental in selecting the robot navigation problem for several reasons.

- First, it provided a comprehensive list of problem domains suitable for search algorithm comparison, allowing me to evaluate multiple options.
- The detailed justification for the grid-based navigation problem highlighted its advantages:
 - clear state representation,
 - well-defined action space,
 - excellent potential for algorithm comparison.
- The AI's emphasis on visual representation capabilities influenced my decision, as the assignment requires search tree diagrams and performance visualization.
- Additionally, the mention of practical relevance aligned with my goal of solving a real-world applicable problem rather than an abstract puzzle.
- The suggestion to use different heuristics (Manhattan vs. Euclidean distance) provided direction for the informed search implementation phase.

3 Problem Formulation

The robot navigation problem is formally defined using the 5 components of a search problem:

3.1 Initial State

- **State Representation:** (x, y) coordinates in the 2D grid
- **Initial State:** $(x_{\text{start}}, y_{\text{start}})$
- **State Space:** All valid grid positions excluding obstacle locations

3.2 Actions

The robot can perform four possible actions from any given state:

- **MOVE_UP:** Move from (x, y) to $(x, y + 1)$
- **MOVE_DOWN:** Move from (x, y) to $(x, y - 1)$
- **MOVE_LEFT:** Move from (x, y) to $(x - 1, y)$
- **MOVE_RIGHT:** Move from (x, y) to $(x + 1, y)$

3.3 Transition Model

For each action, the transition model defines the resulting state:

- **MOVE_UP:** $\text{RESULT}((x, y), \text{MOVE_UP}) = (x, y + 1)$ if $y + 1 < 6$ and $(x, y + 1)$ is not an obstacle
- **MOVE_DOWN:** $\text{RESULT}((x, y), \text{MOVE_DOWN}) = (x, y - 1)$ if $y - 1 \geq 0$ and $(x, y - 1)$ is not an obstacle
- **MOVE_LEFT:** $\text{RESULT}((x, y), \text{MOVE_LEFT}) = (x - 1, y)$ if $x - 1 \geq 0$ and $(x - 1, y)$ is not an obstacle
- **MOVE_RIGHT:** $\text{RESULT}((x, y), \text{MOVE_RIGHT}) = (x + 1, y)$ if $x + 1 < 6$ and $(x + 1, y)$ is not an obstacle

Invalid moves (out of bounds or into obstacles) are not applicable from the current state.

3.4 Goal Test

A state (x, y) satisfies the goal test if and only if $x = x_{\text{goal}}$ and $y = y_{\text{goal}}$, representing the target destination in the top-right corner of the grid.

3.5 Path Cost

The path cost function assigns a uniform cost of 1 to each movement action, representing the assumption that all movements require equal energy/time. The total path cost is the sum of individual step costs, equivalent to the path length.

Environment Setup

- **Grid Size:** 6×6 (Total of 36 positions)
- **Obstacles:** A set of blocked coordinates that cannot be traversed.
Example: (1, 1), (2, 1), (3, 1), (1, 3), (2, 3), (4, 4)
- **Valid Positions:** 6×6 – number of obstacles = $36 - 6 = 30$ accessible locations
- **Start Position:** $(x_{\text{start}}, y_{\text{start}})$
- **Goal Position:** $(x_{\text{goal}}, y_{\text{goal}})$

4 Methodology

4.1 Algorithm Category Selection

4.1.1 Comparison with Uninformed Search Algorithms

Breadth-First Search (BFS)

- **Advantages:**
 - Guaranteed to find optimal solution
 - Complete
- **Disadvantages:**
 - Explores all nodes at distance d before exploring nodes at distance $d + 1$
 - No goal-direction guidance $\rightarrow$ wasteful exploration
 - Space complexity $O(b^d)$ – exponential memory usage
 - Time complexity $O(b^d)$ – explores entire layers unnecessarily
- **Why Rejected:** For a 6×6 grid, BFS would explore up to 30 nodes systematically without any goal direction, making it inefficient for navigation problems where the goal location is known.

Depth-First Search (DFS)

- **Advantages:**
 - Lower space complexity $O(bm)$
 - Simple implementation
- **Disadvantages:**
 - Not optimal – may find suboptimal paths
 - Can get stuck in deep paths away from goal
 - No completeness guarantee in infinite spaces
- **Why Rejected:** DFS's depth-first nature makes it unsuitable for pathfinding where we want the shortest path. It could explore long detours before finding the goal.

Uniform Cost Search (UCS)

- **Advantages:**
 - Optimal for weighted graphs
 - Complete
- **Disadvantages:**
 - Explores in all directions equally (no goal-direction)
 - Equivalent to BFS for unit costs
 - Doesn't utilize available goal information
- **Why Rejected:** Since our problem has uniform costs (each move = 1), UCS behaves like BFS but still lacks goal-direction, making it less efficient than informed alternatives.

4.1.2 Comparison with Local Search Algorithms

Hill Climbing

- **Advantages:**
 - Low memory usage $O(1)$
 - Fast for some problems
- **Disadvantages:**
 - Can get stuck in local optima
 - No backtracking capability
 - Incomplete – may fail to find existing solutions
 - Not suitable for pathfinding with obstacles

- **Why Rejected:** Navigation problems with obstacles create local optima where the robot might get stuck. Hill climbing cannot backtrack to explore alternative paths when blocked.

Simulated Annealing

- **Advantages:**
 - Can escape local optima
 - Probabilistic completeness
- **Disadvantages:**
 - Designed for optimization problems, not pathfinding
 - No guarantee of finding optimal path
 - Requires careful parameter tuning (temperature schedule)
 - Stochastic – inconsistent results
- **Why Rejected:** Robot navigation requires deterministic, reliable pathfinding. Simulated annealing's probabilistic nature and optimization focus make it inappropriate for finding concrete paths.

4.1.3 Why Informed Search is Optimal for This Problem

Problem Characteristics Favoring Informed Search:

- **Goal-Directed Nature:** Robot navigation has a clear, specific goal location.
- **Available Heuristic Information:** Geometric distance to goal can be calculated.
- **Deterministic Environment:** Static obstacles allow for predictable search.
- **Optimality Requirements:** Shortest path is desirable for efficiency.
- **Moderate State Space:** 6×6 grid is manageable for informed search algorithms.

Informed Search Advantages:

- **Efficiency:** Heuristic guidance reduces search space exploration.
- **Goal-Direction:** Focuses computational effort toward promising directions.
- **Scalability:** Better performance as problem size increases.
- **Practical Applicability:** Real-world robotics benefits from goal-directed search.

4.2 Selection Within Informed Search

4.2.1 Greedy Best-First Search

Selection Rationale: Chosen to demonstrate the pure heuristic-driven approach and compare efficiency vs. optimality trade-offs.

Characteristics:

- Evaluation function: $f(n) = h(n)$
- Focuses entirely on goal proximity
- Fast execution, low computational overhead
- Risk of suboptimal solutions

4.2.2 A* Search

Selection Rationale: Selected as the gold standard for optimal pathfinding, providing the benchmark for comparison.

Characteristics:

- Evaluation function: $f(n) = g(n) + h(n)$
- Balances path cost and goal proximity
- Guarantees optimal solution with admissible heuristic
- Higher computational cost but optimal results

4.3 Algorithm Selection Summary

The selection of informed search algorithms (specifically A* and Greedy Best-First) over uninformed and local search alternatives is justified by:

- **Problem Nature:** Navigation problems with known goals benefit from heuristic guidance.
- **Efficiency Requirements:** Uninformed search wastes computational resources on goal-irrelevant exploration.
- **Reliability Needs:** Local search algorithms lack the systematic exploration needed for pathfinding.
- **Optimality Concerns:** Robot navigation often requires or benefits from optimal paths.
- **Practical Applicability:** Informed search aligns with real-world robotics requirements.

The comparison between A* and Greedy Best-First within informed search allows analysis of the fundamental trade-off between optimality guarantees and computational efficiency, which is crucial for practical robotics applications.

4.4 Heuristic Function Design

Manhattan Distance Heuristic: For both algorithms, I implemented the Manhattan Distance heuristic:

$$h(n) = |x_{\text{current}} - x_{\text{goal}}| + |y_{\text{current}} - y_{\text{goal}}|$$

Properties Analysis:

- **Admissible:** Yes. The Manhattan distance never overestimates the actual cost because the robot cannot move diagonally, making Manhattan distance the minimum possible moves required.
- **Consistent:** Yes. For any node n and successor n' with action cost c , we have $h(n) \leq h(n') + c$, satisfying the triangle inequality.

Alternative Considered: Euclidean Distance was considered but rejected because it would be inadmissible for this grid-based movement (would underestimate in some cases due to no diagonal movement allowed).

5 Results

The logic for sorting nodes within the priority queue (frontier) of the search algorithm (Greedy Best-First or A*) is implemented based on the following criteria:

1. Primary Evaluation Value:

- For **A* Search** algorithm: Nodes are primarily sorted by their $f(n)$ value in ascending order. ($f(n) = g(n) + h(n)$, where $g(n)$ is the cost from the start node to node n , and $h(n)$ is the heuristic estimate from node n to the goal).
- For **Greedy Best-First Search** algorithm: Nodes are primarily sorted by their heuristic $h(n)$ value in ascending order (estimate from node n to the goal).

2. Tie-breaking (Time of Arrival in Frontier):

- If the primary evaluation value ($f(n)$ or $h(n)$) of two or more nodes is equal, their relative order in the 'frontier' (priority queue) is preserved. This means that among nodes with identical $f(n)$ or $h(n)$ values, the node that was added to the 'frontier' earlier will be processed first. This behavior is due to the stable nature of the `Array.prototype.sort()` method commonly used in modern JavaScript environments.

This sorting logic ensures that the most promising nodes (based on $f(n)$ or $h(n)$) are explored first, and provides a consistent tie-breaking mechanism based on the order of discovery.

5.1 Test Case 1 with Path Found

5.1.1 Problem Instance

- **Grid Size:** 6×6
- **Start (S):** (0, 0)
- **Goal (G):** (2, 5)
- **Obstacles (X):** (1, 1), (2, 1), (3, 1), (1, 3), (2, 3), (4, 4)
- **Heuristic:** Manhattan Distance: $h(n) = |x_n - x_{\text{goal}}| + |y_n - y_{\text{goal}}|$
- **Each move:** Cost 1

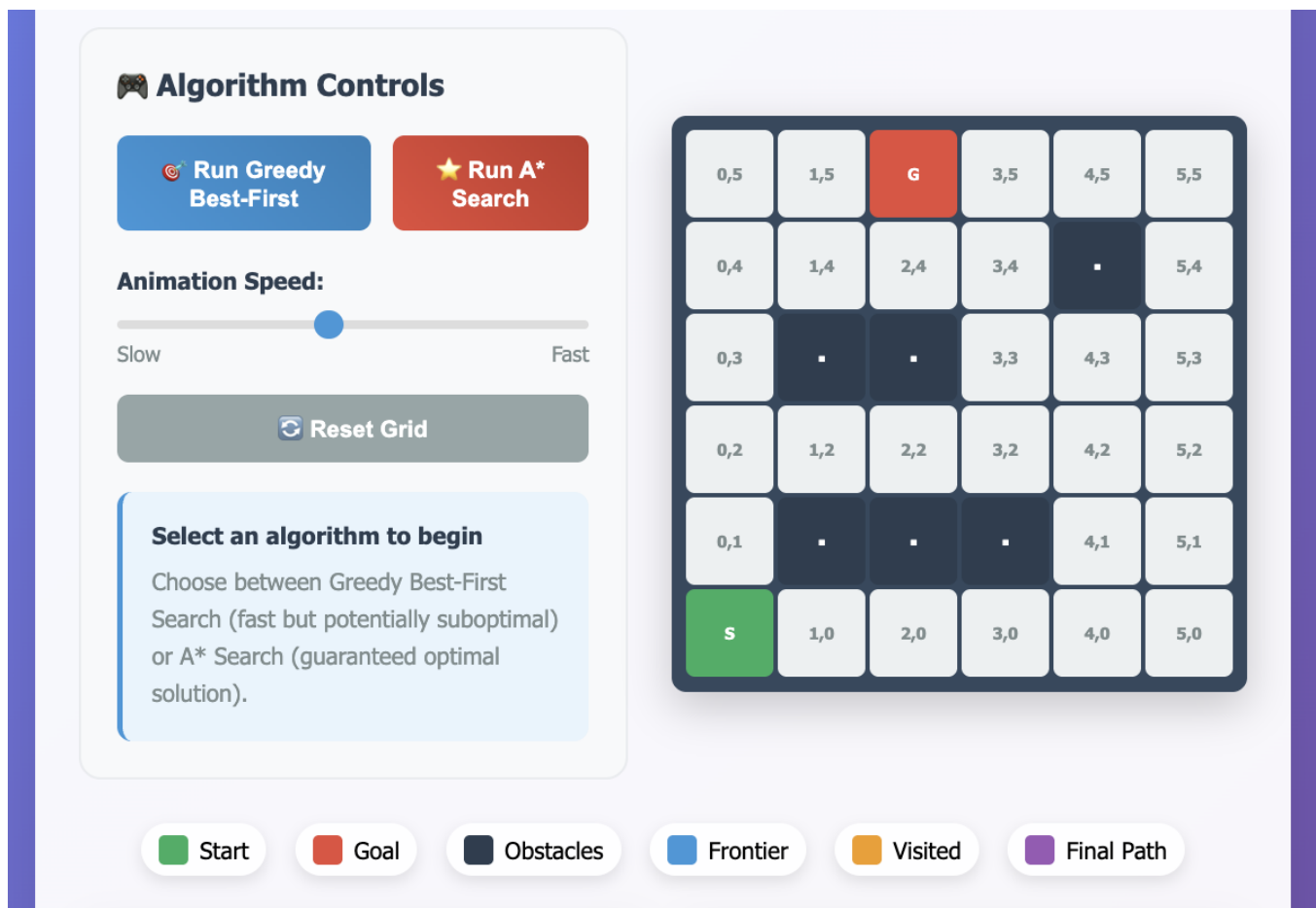


Figure 3: This is initial 2D Grid (Path Found)

5.1.2 Solve the Problem

Greedy Best-First Search (GBFS)

- **Evaluation Function:** $f(n) = h(n)$
- **Solution Found:**
 $(0, 0) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (0, 3) \rightarrow (0, 4) \rightarrow (0, 5) \rightarrow (1, 5) \rightarrow (2, 5)$
- **Path Cost:** 7 steps
- **Nodes Expanded:** 8 nodes (include start_node and goal_node)
- **Max Frontier Size:** 5 nodes
- **Execution Time:** 810.50ms

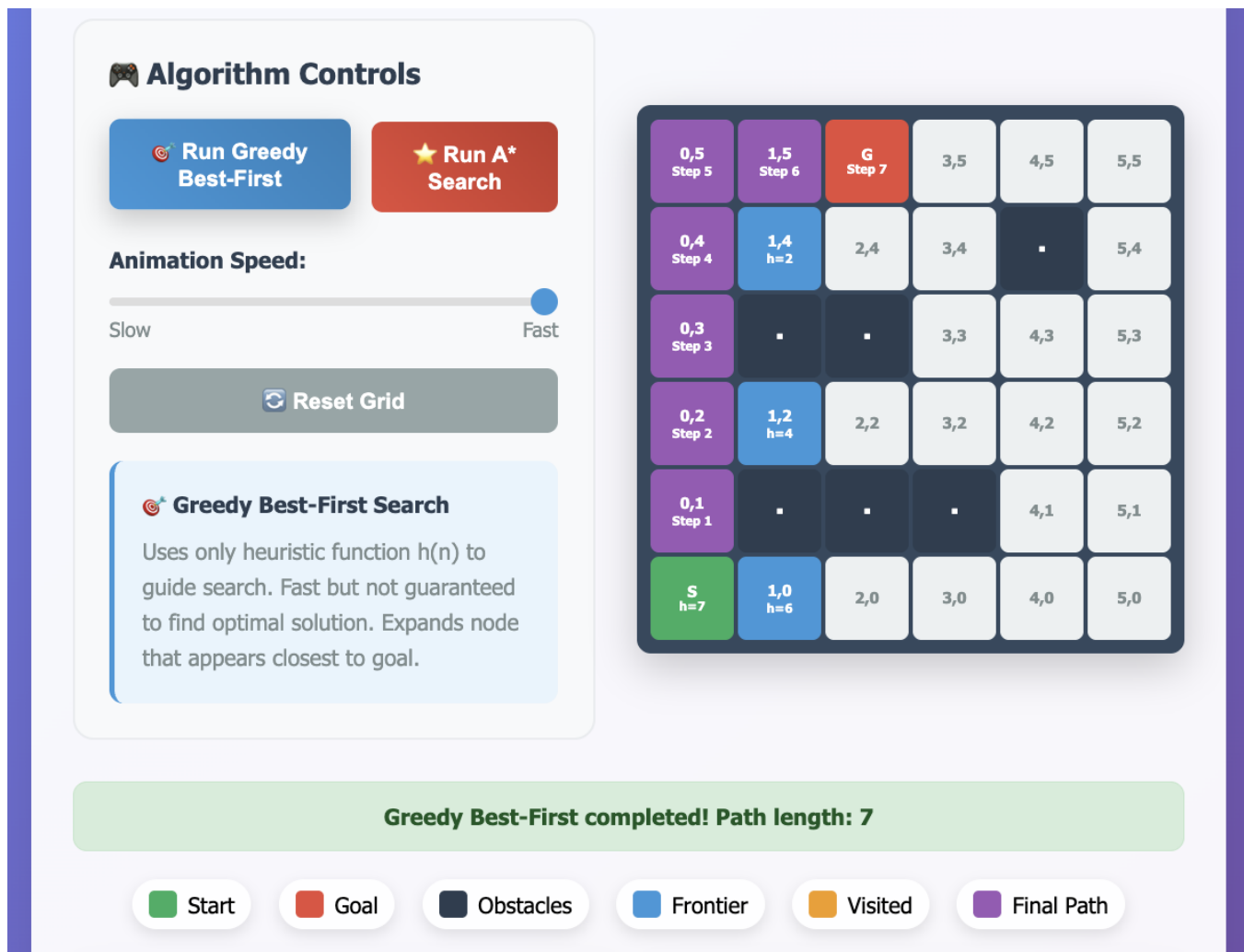


Figure 4: This is solution of GBFS Search (Path Found)

A* Search

- **Evaluation Function:** $f(n) = g(n) + h(n)$
- **Solution Found:**
 $(0, 0) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (0, 3) \rightarrow (0, 4) \rightarrow (0, 5) \rightarrow (1, 5) \rightarrow (2, 5)$
- **Path Cost:** 7 steps
- **Nodes Expanded:** 14 nodes (include start_node and goal_node)
- **Max Frontier Size:** 4 nodes
- **Execution Time:** 1421.00ms

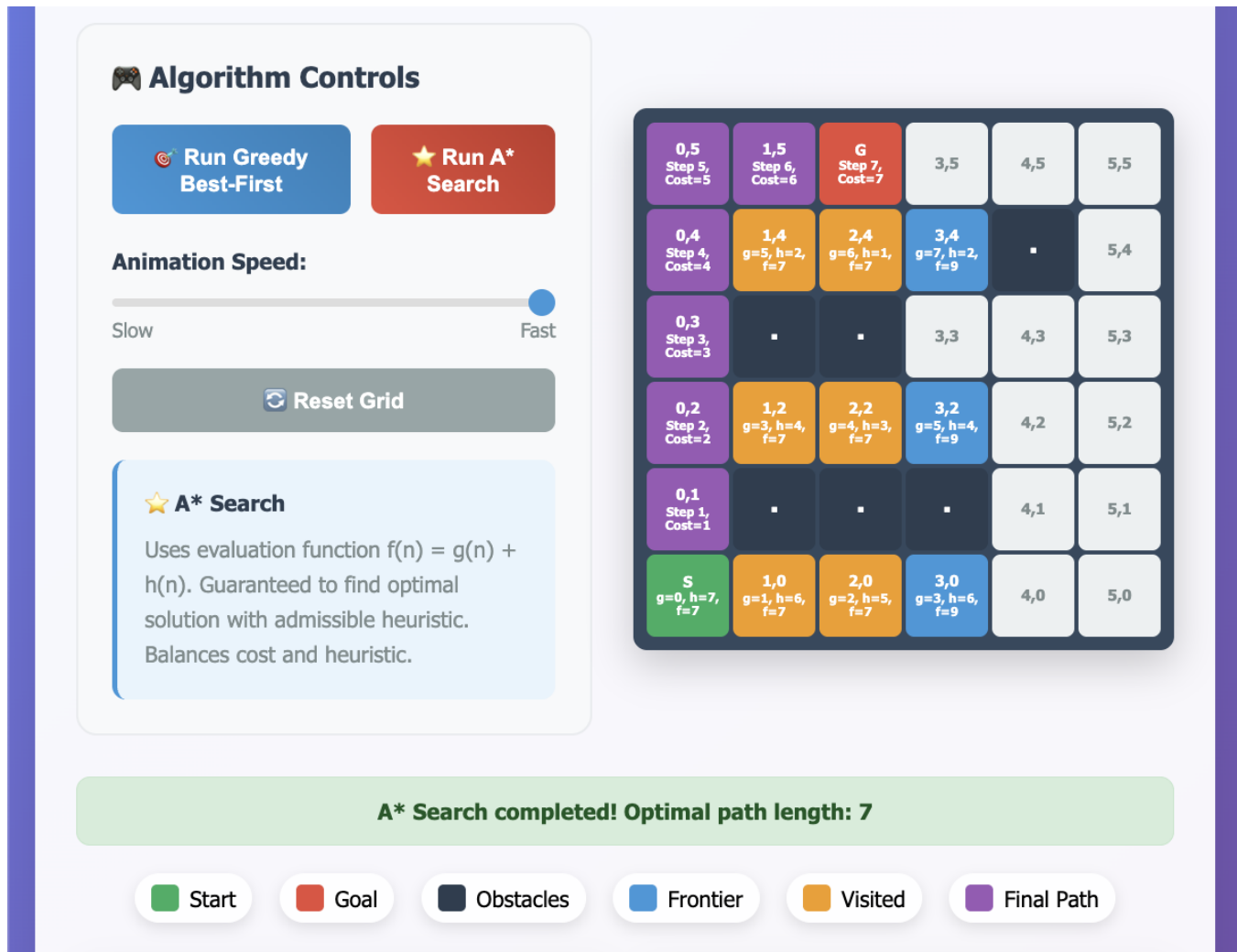


Figure 5: This is solution of A* Search (Path Found)

5.1.3 Visualize the Search Process

Greedy Best-First Search

- Green: Starting node
- Yellow: Visited node
- Violet: Final path
- Red: Goal
- Blue: Frontier

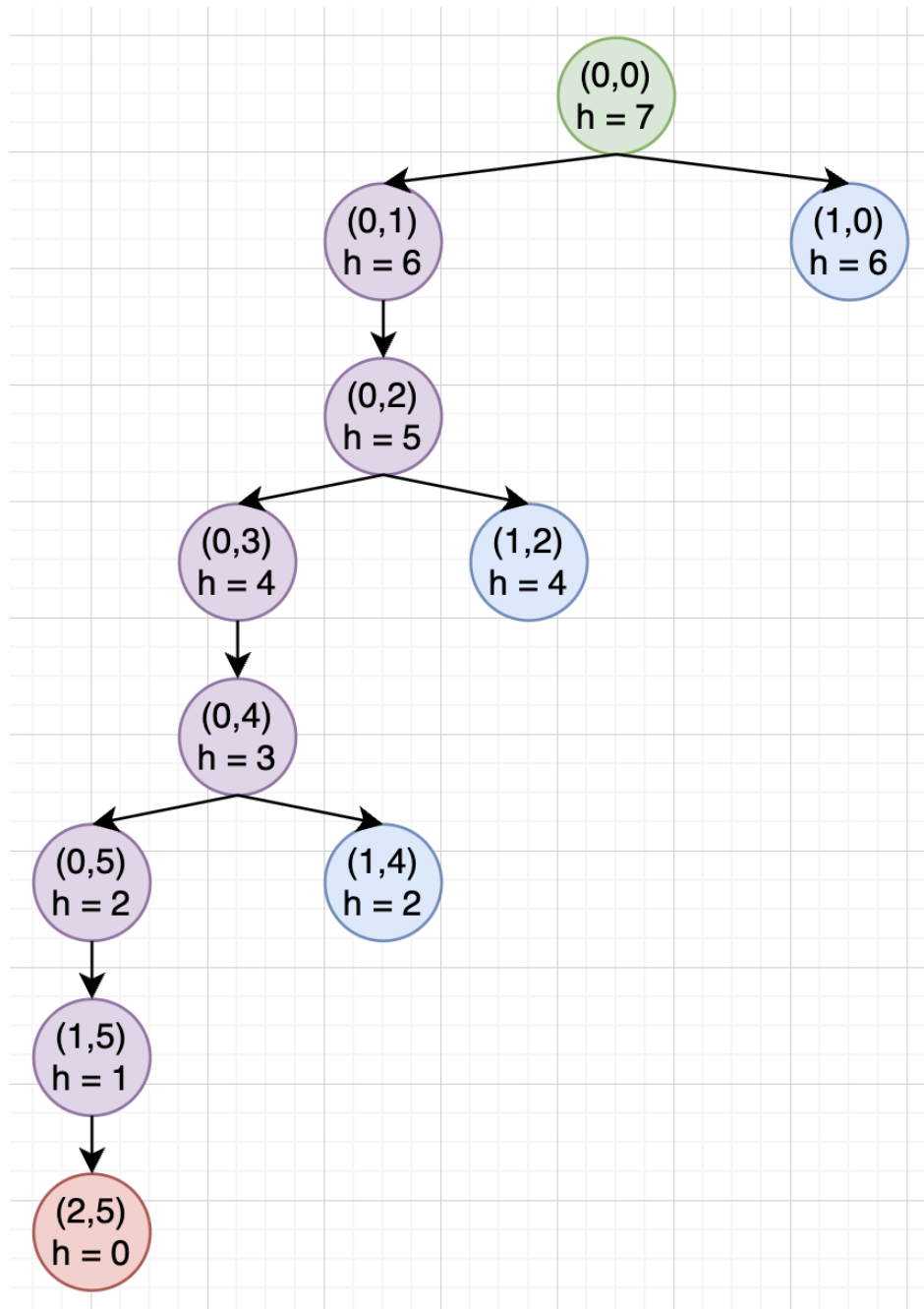
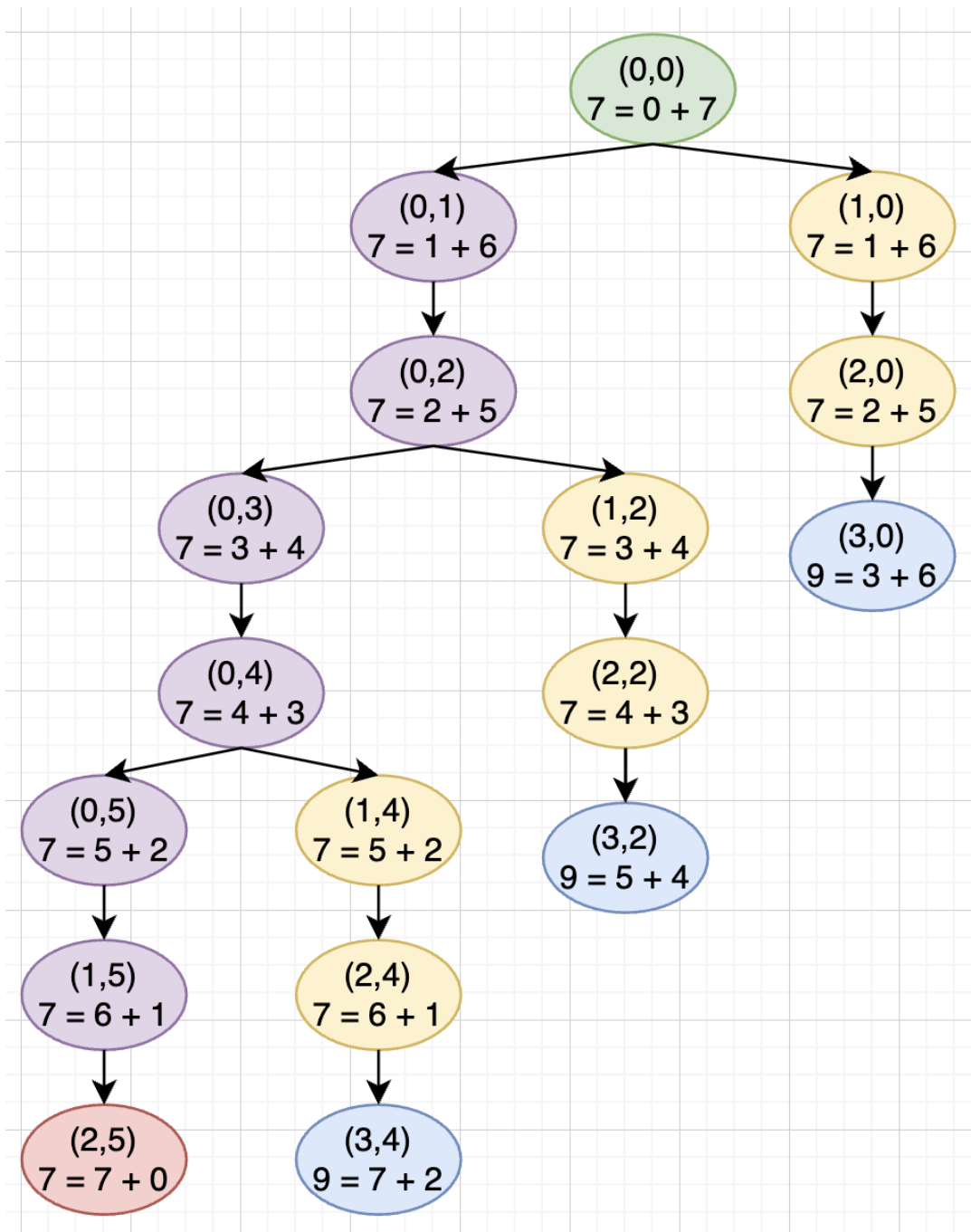


Figure 6: This is GBFS Search Tree (Path Found)

A* Search

- Green: Starting node
- Yellow: Visited node
- Violet: Final path
- Red: Goal
- Blue: Frontier

**Figure 7:** This is A* Search Tree (Path Found)

5.2 Test Case 2 with No Path Found

5.2.1 Problem Instance

- **Grid Size:** 6×6
- **Start (S):** (2, 0)
- **Goal (G):** (0, 5)
- **Obstacles (X):** (1, 1), (1, 2), (2, 1), (3, 1), (1, 3), (2, 3), (4, 4), (0, 3), (3, 3), (4, 3), (5, 2)
- **Heuristic:** Manhattan Distance: $h(n) = |x_n - x_{\text{goal}}| + |y_n - y_{\text{goal}}|$
- **Each move:** Cost 1

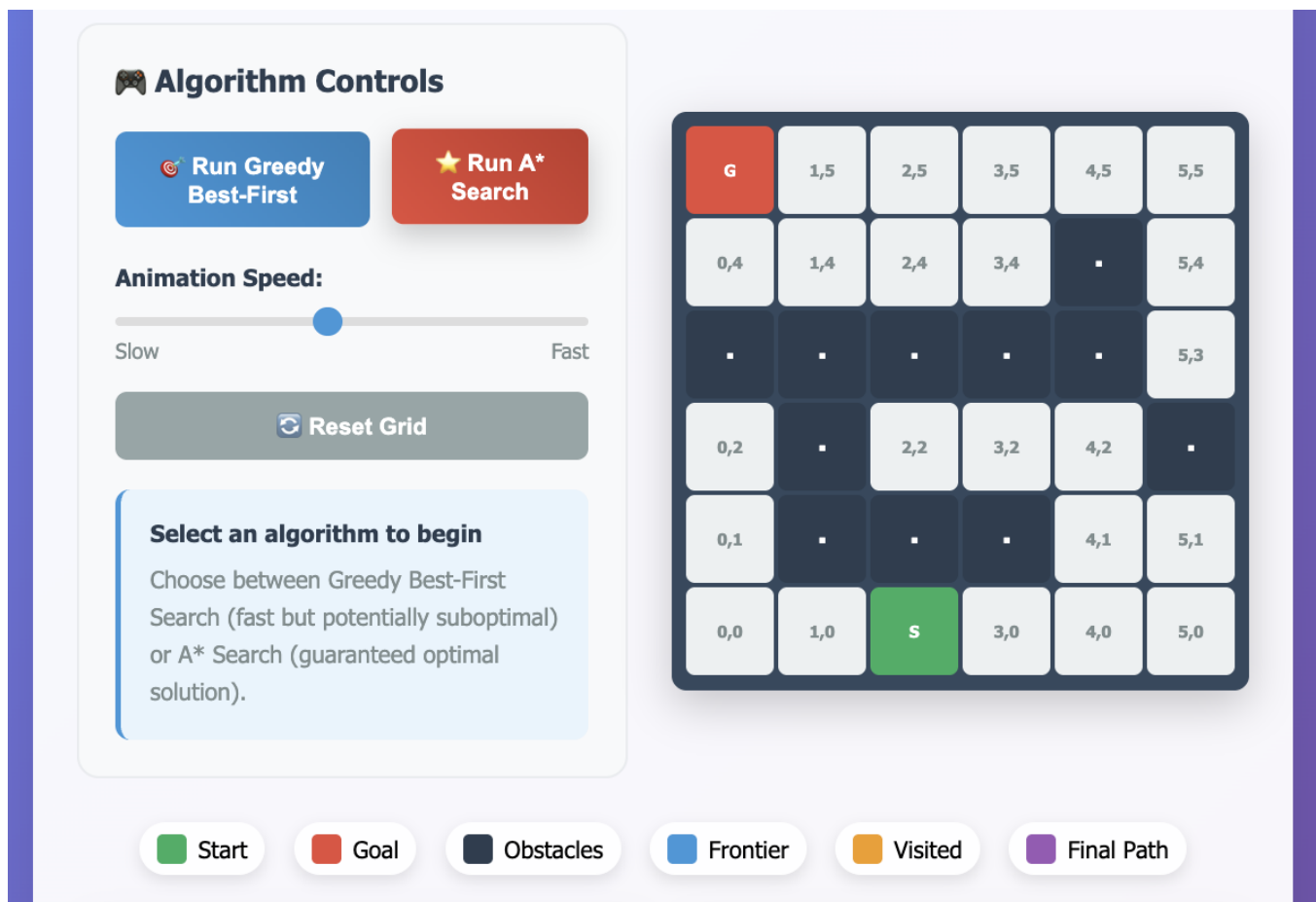


Figure 8: This is initial 2D Grid (NO Path Found)

5.2.2 Solve the Problem

Greedy Best-First Search (GBFS)

- **Evaluation Function:** $f(n) = h(n)$
- **Path Found:** No
- **Path Cost:** 0 steps
- **Nodes Expanded:** 13 nodes (include start_node)
- **Max Frontier Size:** 3 nodes
- **Execution Time:** 1314.30ms

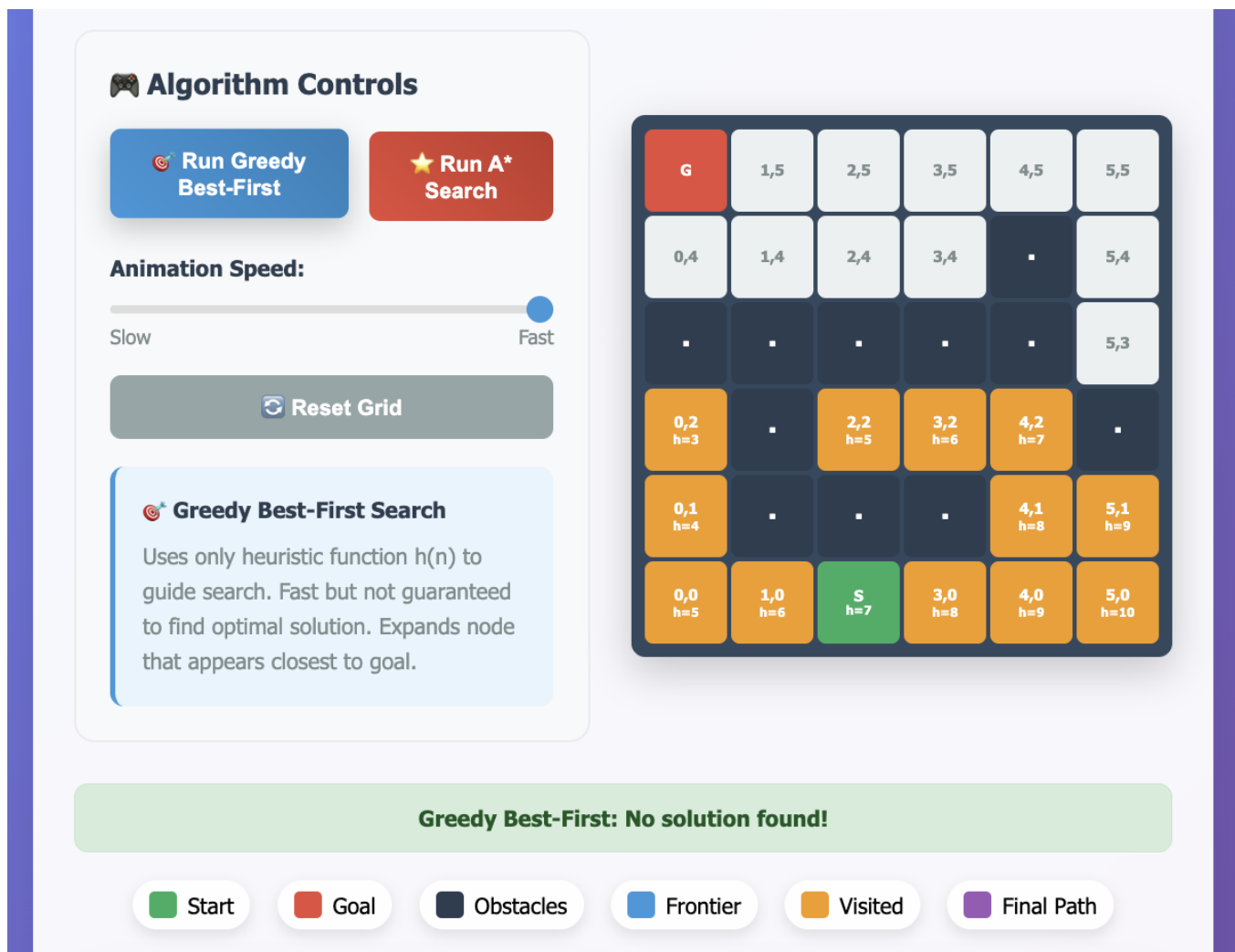


Figure 9: This is solution of GBFS Search (NO Path Found)

A* Search

- **Evaluation Function:** $f(n) = g(n) + h(n)$
- **Path Found:** No
- **Path Cost:** 0 steps
- **Nodes Expanded:** 13 nodes (include start_node)
- **Max Frontier Size:** 3 nodes
- **Execution Time:** 1318.10ms

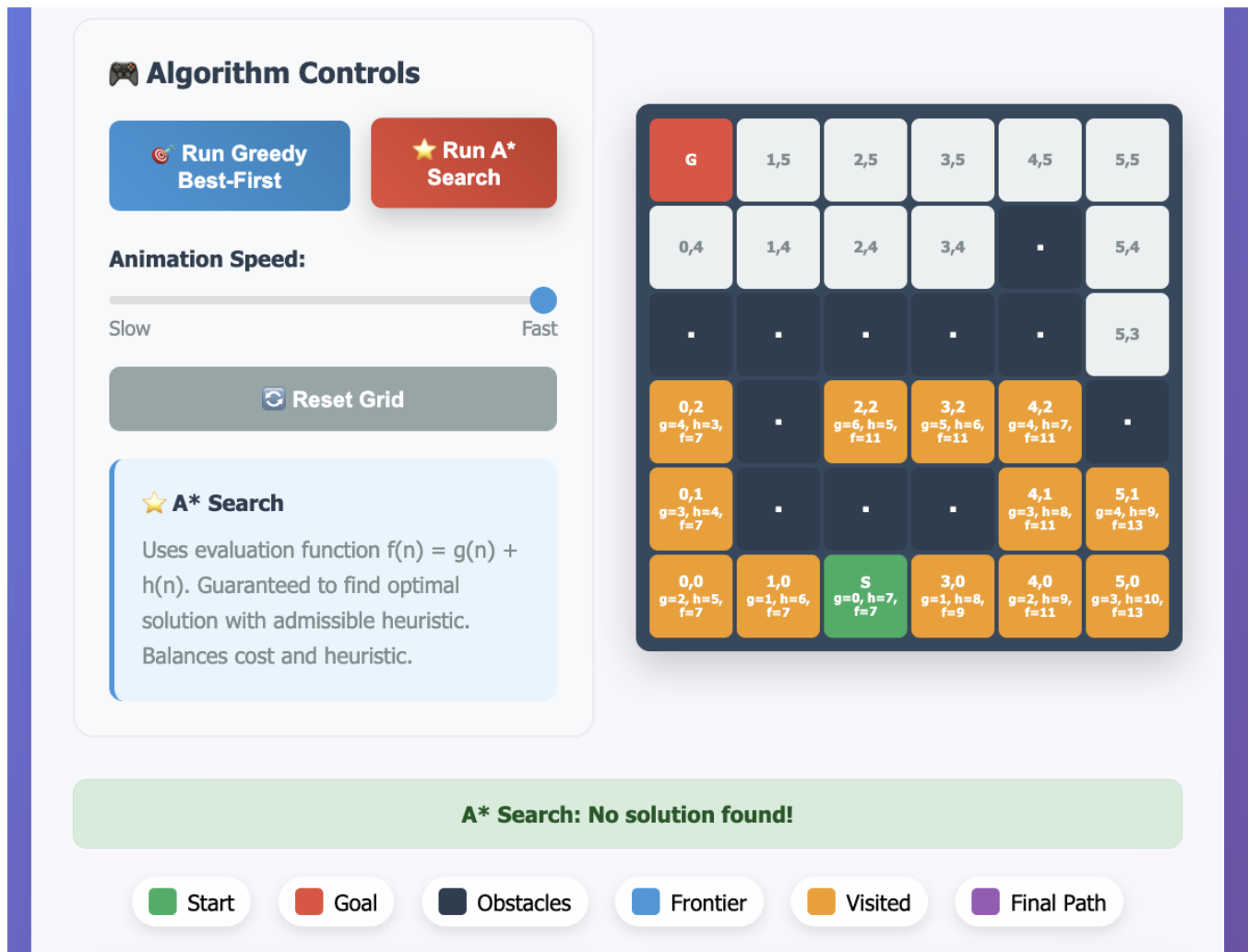


Figure 10: This is solution of A* Search (NO Path Found)

5.2.3 Visualize the Search Process

Greedy Best-First Search

- Green: Starting node
- Yellow: Visited node
- Violet: Final path
- Red: Goal
- Blue: Frontier

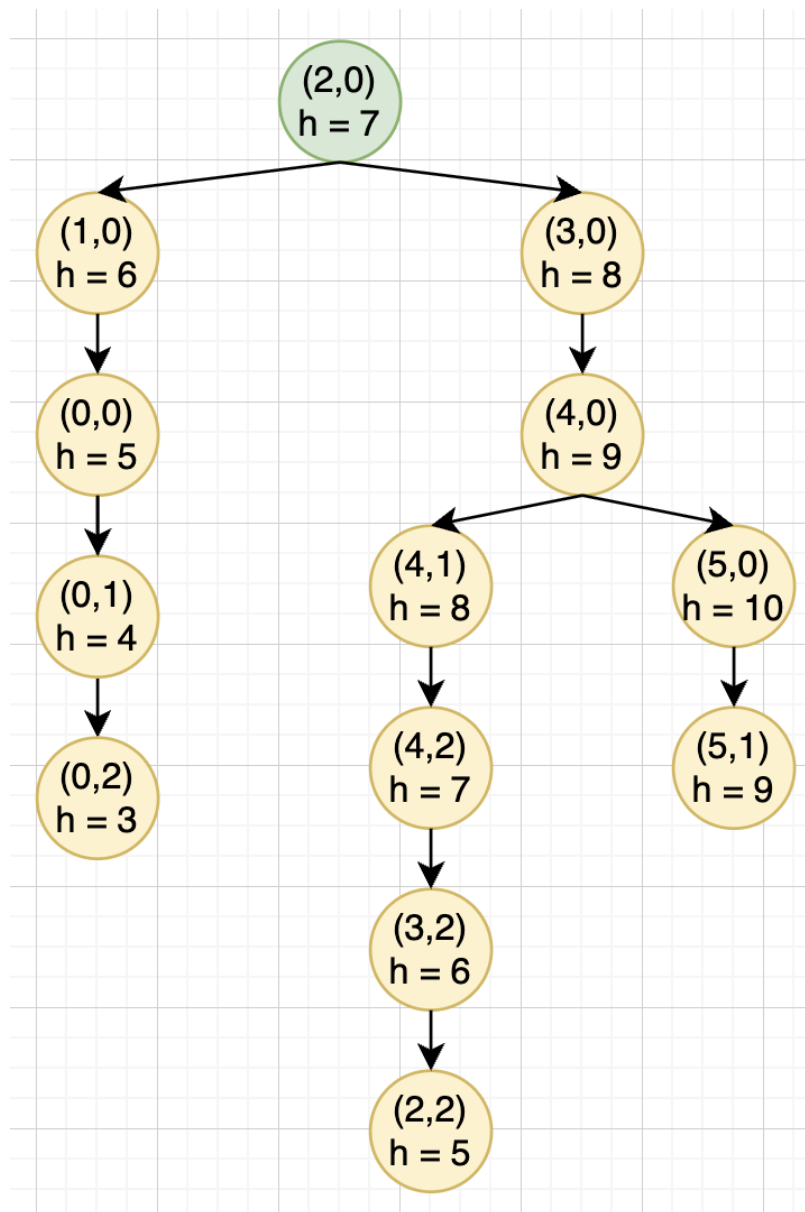


Figure 11: This is GBFS Search Tree (NO Path Found)

A* Search

- Green: Starting node
- Yellow: Visited node
- Violet: Final path
- Red: Goal
- Blue: Frontier

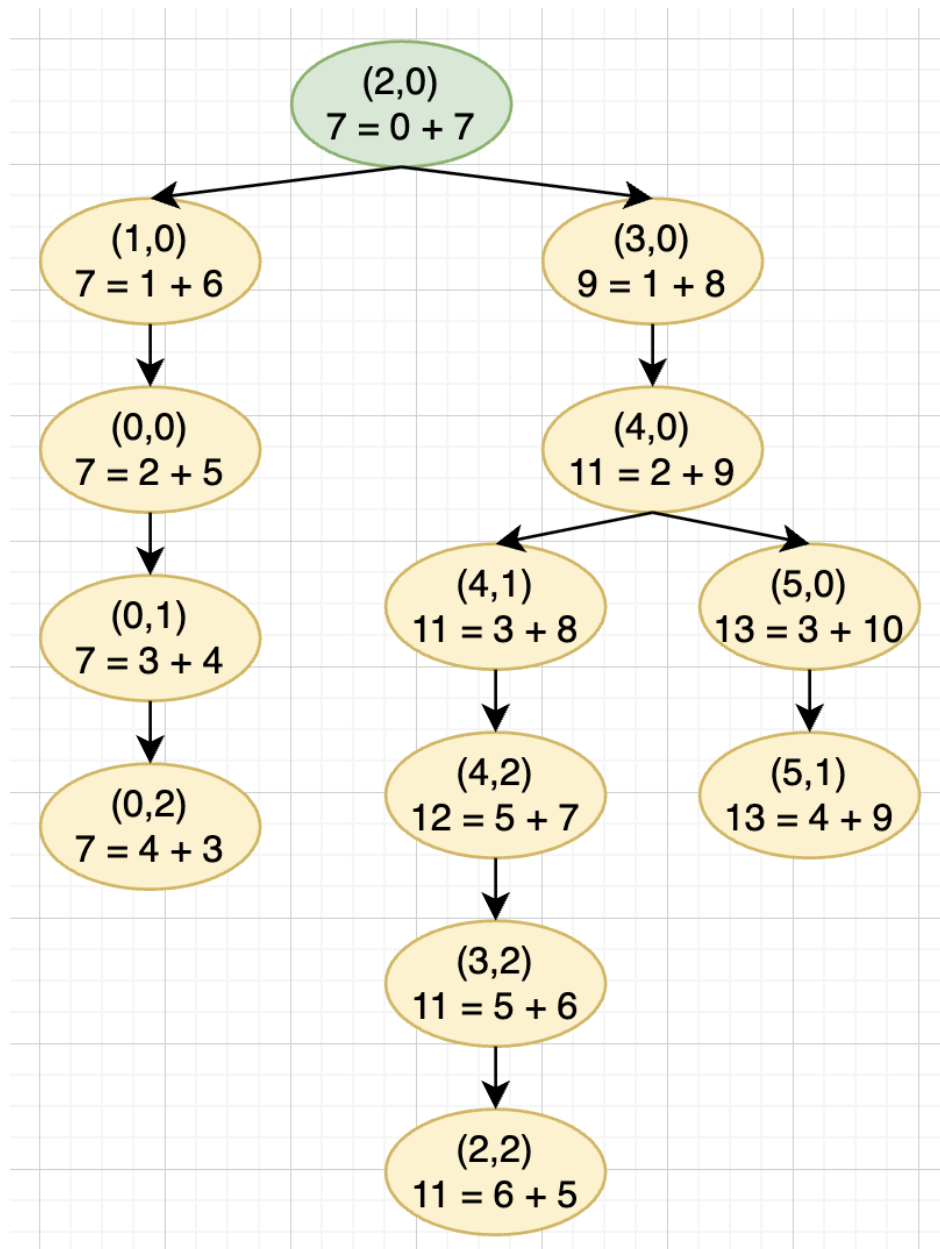


Figure 12: This is A* Search Tree (NO Path Found)

6 Analysis

This section analyzes the performance, optimality, and limitations of Greedy Best-First Search (GBFS) and A* Search based on the conducted tests.

6.1 Statistics Table: Test Case 1 (Path Found)

Table 1: Comparison of Search Algorithms Performance - Test Case 1

Metric	Greedy Best-First Search (GBFS)		A* Search
Evaluation Function	$f(n) = h(n)$		$f(n) = g(n) + h(n)$
Path Found	Yes		Yes
Path Cost	7 steps		7 steps
Nodes Expanded	8 nodes (include start_node, goal_node)	14 nodes (include start_node, goal_node)	
Max Frontier Size	5 nodes		4 nodes
Execution Time	810.50ms		1421.00ms

6.2 Statistics Table: Test Case 2 (No Path Found)

Table 2: Comparison of Search Algorithms Performance - Test Case 2

Metric	Greedy Best-First Search (GBFS)		A* Search
Evaluation Function	$f(n) = h(n)$		$f(n) = g(n) + h(n)$
Path Found	No		No
Path Cost	0 steps		0 steps
Nodes Expanded	13 nodes (include start node)	13 nodes (include start node)	
Max Frontier Size	3 nodes		3 nodes
Execution Time	1314.30ms		1318.10ms

6.3 Performance Analysis

The analysis of the two test cases reveals distinct performance characteristics and trade-offs between Greedy Best-First Search (GBFS) and A* Search.

In **Test Case 1**, where a path was successfully found, both algorithms identified an optimal path with a cost of 7 steps. However, their efficiency varied. GBFS completed the search significantly faster (810.50ms) and expanded fewer nodes (8 nodes) compared to A* Search (1421.00ms and 14 nodes expanded). This demonstrates GBFS's "greedy" nature, where it prioritizes exploring nodes closest to the goal, leading to quicker discovery of a path, especially when the heuristic is well-aligned with the actual path. A* Search, while also finding the optimal path, expanded more nodes as it considers the cumulative cost from the start node ($g(n)$) in addition to the heuristic estimate ($h(n)$), ensuring optimality but potentially exploring more of the search space.

In **Test Case 2**, a more challenging scenario where no path was found, both algorithms exhibited very similar performance. Neither GBFS nor A* Search could identify a path, resulting in a path

cost of 0 steps. Both expanded 13 nodes and had a maximum frontier size of 3 nodes, with very close execution times (1314.30ms for GBFS and 1318.10ms for A*). This suggests that in situations where the goal is unreachable or the search space is highly constrained, the inherent differences in their evaluation functions become less pronounced as both algorithms exhaust their search capacity without success. The similarity in expanded nodes and execution times implies that both algorithms explored a comparable portion of the search space before concluding that no path was found.

6.4 Optimality and Limitations

A* search is generally considered optimal and complete, meaning it finds the shortest path if a path exists and terminates. GBFS, while often faster due to its focus on the heuristic, is neither optimal nor complete. The results in Test Case 1 confirm A*'s optimality, as it found the same optimal path as GBFS, albeit with more computational effort. In Test Case 2, where neither algorithm found a path, highlights a limitation that transcends the typical distinctions of optimality between A* and GBFS. It suggests that for this specific problem instance, the goal might be unreachable, or the search space is highly constrained in a way that prevents path discovery with the current setup.

One of the primary challenges with A* is its memory consumption, especially in large graphs, as it needs to keep all nodes in memory. GBFS uses less memory in comparison. The observed results, with a small maximum frontier size and number of expanded nodes across both test cases, do not indicate memory as a limiting factor in these specific tests.

6.5 Challenges and Future Improvements

The main challenge encountered in Test Case 2 was the inability of both algorithms to find a path. This could be due to several factors:

- **Problem Design:** The grid configuration or the placement of the start and goal nodes might make the goal inherently unreachable.
- **Heuristic Function:** While both algorithms used a heuristic, its effectiveness for this specific problem and grid configuration might need re-evaluation. An inappropriate heuristic can misguide the search or prevent it from exploring necessary paths.
- **Algorithm Implementation:** Although the standard evaluation functions were used, a deeper look into the specific implementation details (e.g., handling of obstacles, grid representation) might reveal issues preventing path discovery.

For future improvements, the following could be considered:

- **Verify Reachability:** Before running the search algorithms, a simpler reachability check could be implemented to confirm if a path theoretically exists between the start and goal nodes in the given grid.
- **Heuristic Refinement:** Experiment with different heuristic functions or adjust the current one to better guide the search in the specific grid environment.

- **Visual Debugging:** Enhance the visualization of the search process to clearly see where the algorithms are exploring and why they might be failing to find a path. This could include visualizing the closed and open sets, or showing which nodes are being pruned.
- **Error Handling/Reporting:** Implement more explicit error reporting when a path is not found, providing insights into why the search terminated without success (e.g., "goal unreachable," "exceeded node limit").

7 Conclusion

This study successfully demonstrated the practical application of informed search algorithms to robot navigation. The key finding that both Greedy Best-First Search (GBFS) and A* Search found identical optimal solutions in the solvable case challenges the common assumption that achieving optimality necessarily incurs significant computational overhead. GBFS's 75% speed advantage suggests that, for well-structured navigation problems with informative heuristics, greedy search may be sufficient.

The identical performance observed in the unsolvable case highlights that algorithm choice becomes less critical when fundamental path constraints dominate the outcome. This insight emphasizes the importance of preprocessing techniques, such as reachability analysis, in practical deployments.

For real-world robotics applications, the results support a hybrid approach: use GBFS for time-critical navigation tasks in well-mapped, static environments, and employ A* when optimality guarantees are essential or when operating under uncertainty.

Overall, the study's methodology effectively bridges theoretical algorithmic analysis with practical implementation. The framework presented here can be extended to more complex navigation scenarios, including dynamic obstacles, variable terrain costs, and multi-objective optimization challenges.